

Lab 3: Processes, PIDs and Exit Status

Lab Report and Knowledge Check

I. Introduction

This report documents the practical work completed for Lecture 3 of ST5003CMD Operating Systems, Security and Networks. The lab used small C programs to explore how a running program appears to the operating system: launching a program as a background process and inspecting it with `ps`, reading a process's own ID and its parent's ID with `getpid()` and `getppid()`, reporting success or failure to the shell through `main()`'s return value, and reading input from the user with `scanf()`. All five programs were written in nano as `test.c` and compiled with `gcc` into a separate executable for each task (task1 to task5).

II. Task 1 — Running a Program in the Background

A. The Program

The first program prints a start message, loops 30 times pausing for one second on each pass with `sleep()`, and then prints a finish message. This makes it run for roughly 30 seconds, long enough to be observed while it is still alive. `sleep()` is declared in `unistd.h`, so that header is included alongside `stdio.h`.

```
#include <stdio.h>
#include <unistd.h>

int main() {
    printf("I am starting...\n");
    // Loop for 30 seconds
    for(int i = 1; i <= 30; i++) {
        sleep(1); // Pauses execution for 1 second
    }
    printf("I am finished.\n");
    return 0;
}
```



```
GNU nano 8.7.1 test.c
#include <stdio.h>
#include <unistd.h>

int main() {
    printf("I am starting...\n");
    // Loop for 30 seconds
    for(int i = 1; i <= 30; i++) {
        sleep(1); // Pauses execution for 1 second
    }
    printf("I am finished.\n");
    return 0;
}
```

Help Write Out Read File [Read 12 lines] Cut Paste Execute Justify

Figure 1. test.c for Task 1 opened in nano.

B. Compiling and Launching in the Background

The program was compiled to task1 and started with a trailing & so that the shell runs it in the background and immediately returns the prompt. The shell reports the job number and the process ID (PID), here [1] 6715, and the program's first line of output appears while the prompt is already available for more commands.

```
gcc test.c -o task1
./task1 &
```



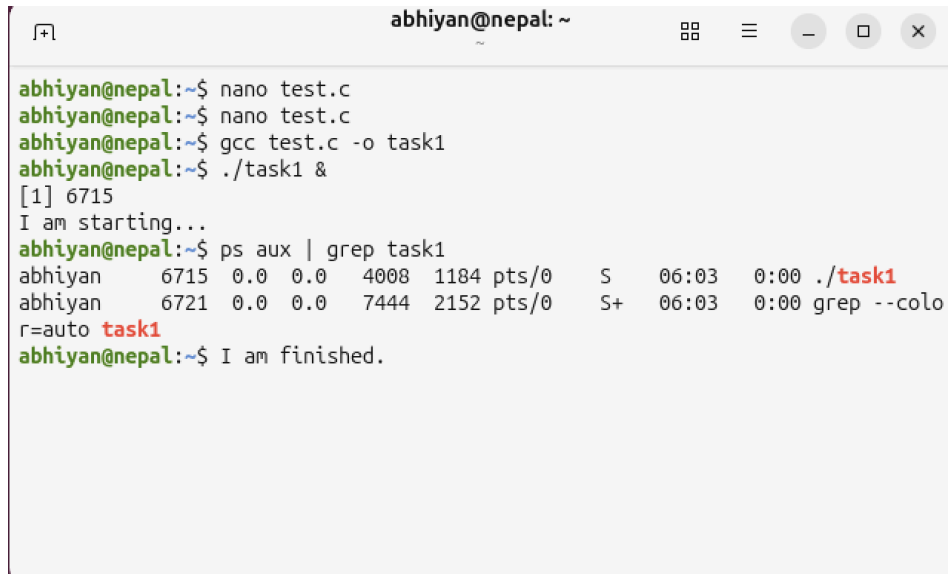
```
abhiyan@nepal: ~
abhiyan@nepal:~$ nano test.c
abhiyan@nepal:~$ nano test.c
abhiyan@nepal:~$ gcc test.c -o task1
abhiyan@nepal:~$ ./task1 &
[1] 6715
I am starting...
abhiyan@nepal:~$
```

Figure 2. Compiling task1 and launching it in the background with &.

C. Inspecting the Process with ps

While task1 was still sleeping, `ps aux | grep task1` was used to find it in the process list. The first line is the program itself: PID 6715, state S (sleeping, because it is inside `sleep()`), running on terminal pts/0, command `./task1`. The second line is the `grep` command, which matches its own search term; its state S+ marks it as a foreground process.

```
ps aux | grep task1
```

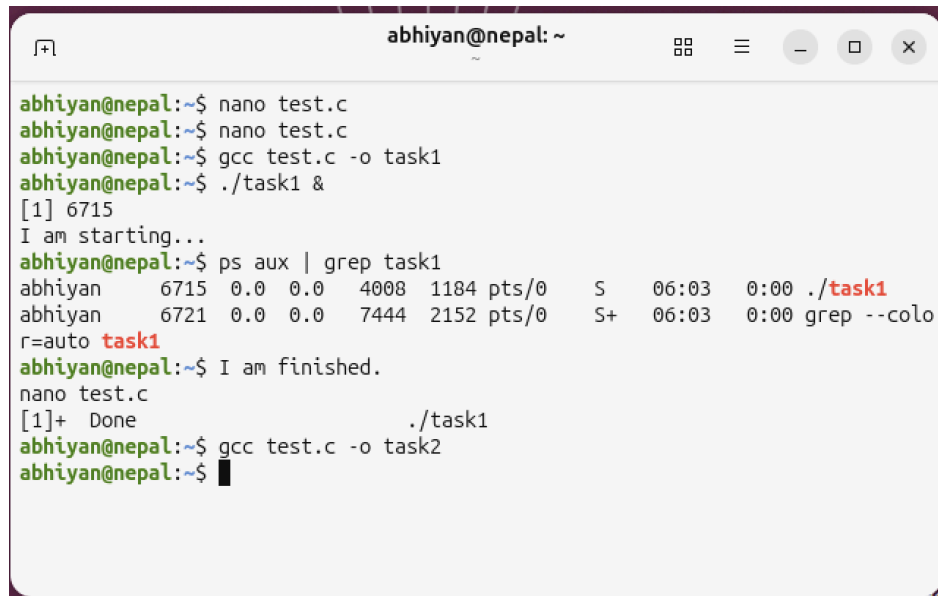
A terminal window titled 'abhiyan@nepal: ~' showing a series of commands and their outputs. The user runs 'nano test.c', 'gcc test.c -o task1', and './task1 &'. The output of './task1 &' is '[1] 6715' followed by 'I am starting...'. Then, the user runs 'ps aux | grep task1', which displays two lines: 'abhiyan 6715 0.0 0.0 4008 1184 pts/0 S 06:03 0:00 ./task1' and 'abhiyan 6721 0.0 0.0 7444 2152 pts/0 S+ 06:03 0:00 grep --colo'. The first line shows the background process in state 'S'. The second line shows the foreground 'grep' process in state 'S+'. Finally, the user enters 'I am finished.'.

```
abhiyan@nepal:~$ nano test.c
abhiyan@nepal:~$ nano test.c
abhiyan@nepal:~$ gcc test.c -o task1
abhiyan@nepal:~$ ./task1 &
[1] 6715
I am starting...
abhiyan@nepal:~$ ps aux | grep task1
abhiyan  6715  0.0  0.0  4008 1184 pts/0    S   06:03   0:00 ./task1
abhiyan  6721  0.0  0.0  7444 2152 pts/0    S+  06:03   0:00 grep --colo
r=auto task1
abhiyan@nepal:~$ I am finished.
```

Figure 3. `ps aux | grep task1` showing the background process with PID 6715 in state S.

D. Completion of the Background Job

After about 30 seconds the program printed “I am finished.” on top of the shell prompt, since the background job writes to the same terminal, and the shell reported `[1]+ Done ./task1` the next time it displayed a prompt.

A terminal window titled 'abhiyan@nepal: ~' showing a sequence of commands and their outputs. The user creates a file 'test.c', compiles it with 'gcc test.c -o task1', and runs it in the background with './task1 &'. The shell returns '[1] 6715' and the program prints 'I am starting...'. The user then runs 'ps aux | grep task1', which shows two entries: one for the user's process (PID 6715, state S) and one for the 'grep' command (PID 6721, state S+). The user enters 'I am finished.' and the shell returns '[1]+ Done ./task1', indicating the background job has completed. The user then runs 'gcc test.c -o task2' and the prompt returns.

```
abhiyan@nepal:~$ nano test.c
abhiyan@nepal:~$ nano test.c
abhiyan@nepal:~$ gcc test.c -o task1
abhiyan@nepal:~$ ./task1 &
[1] 6715
I am starting...
abhiyan@nepal:~$ ps aux | grep task1
abhiyan      6715  0.0  0.0  4008  1184 pts/0    S      06:03   0:00 ./task1
abhiyan      6721  0.0  0.0  7444  2152 pts/0    S+     06:03   0:00 grep --colo
r=auto task1
abhiyan@nepal:~$ I am finished.
nano test.c
[1]+  Done                  ./task1
abhiyan@nepal:~$ gcc test.c -o task2
abhiyan@nepal:~$
```

Figure 4. The background job finishing, reported by the shell as Done.

Summary: task1 was compiled and run as a background job with &. ps aux | grep confirmed it was alive as PID 6715 in the sleeping state, and the shell reported it as Done once the 30 seconds had elapsed.

III. Task 2 — Process ID and Parent Process ID

A. The Program

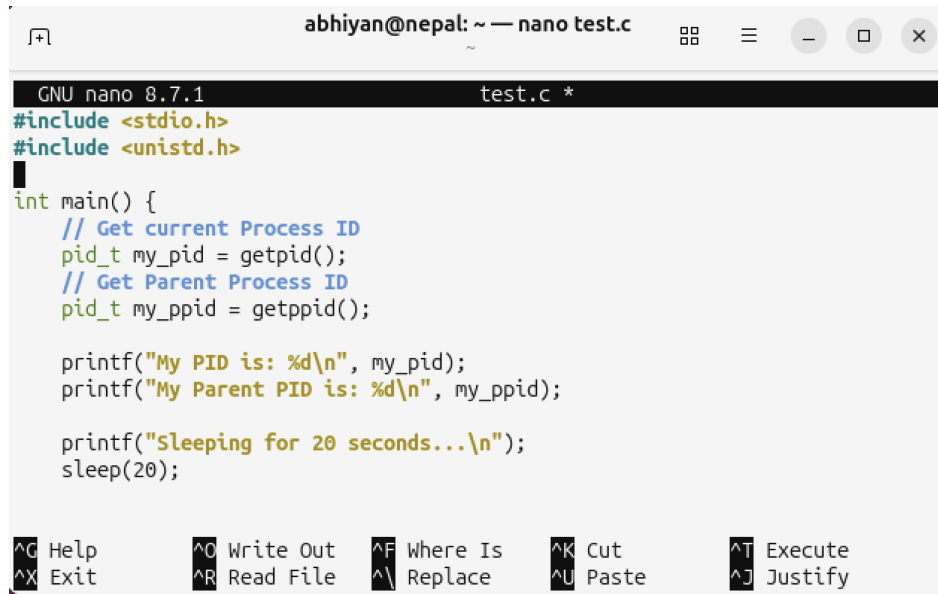
The second program asks the operating system for its own PID with getpid() and its parent's PID with getppid(), prints both, and then sleeps for 20 seconds so the process stays alive long enough to be examined from another command.

```
#include <stdio.h>
#include <unistd.h>

int main() {
    // Get current Process ID
    pid_t my_pid = getpid();
    // Get Parent Process ID
    pid_t my_ppid = getppid();

    printf("My PID is: %d\n", my_pid);
    printf("My Parent PID is: %d\n", my_ppid);

    printf("Sleeping for 20 seconds...\n");
    sleep(20);
    return 0;
}
```



```
abhiyan@nepal: ~ — nano test.c
GNU nano 8.7.1 test.c *
#include <stdio.h>
#include <unistd.h>

int main() {
    // Get current Process ID
    pid_t my_pid = getpid();
    // Get Parent Process ID
    pid_t my_ppid = getppid();

    printf("My PID is: %d\n", my_pid);
    printf("My Parent PID is: %d\n", my_ppid);

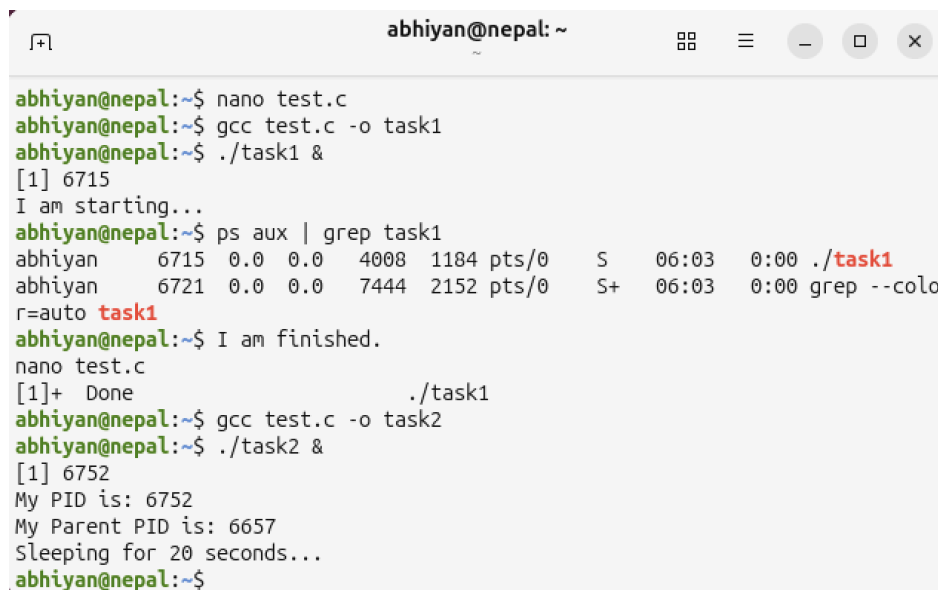
    printf("Sleeping for 20 seconds...\n");
    sleep(20);
}
```

Figure 5. test.c for Task 2 in nano, using getpid() and getppid().

B. Running the Program

task2 was compiled and started in the background. It reported PID 6752 for itself and PID 6657 for its parent. The parent is the shell that launched it, which is why the PPID is the same kind of number that would be seen for any program started from this terminal.

```
gcc test.c -o task2
./task2 &
```



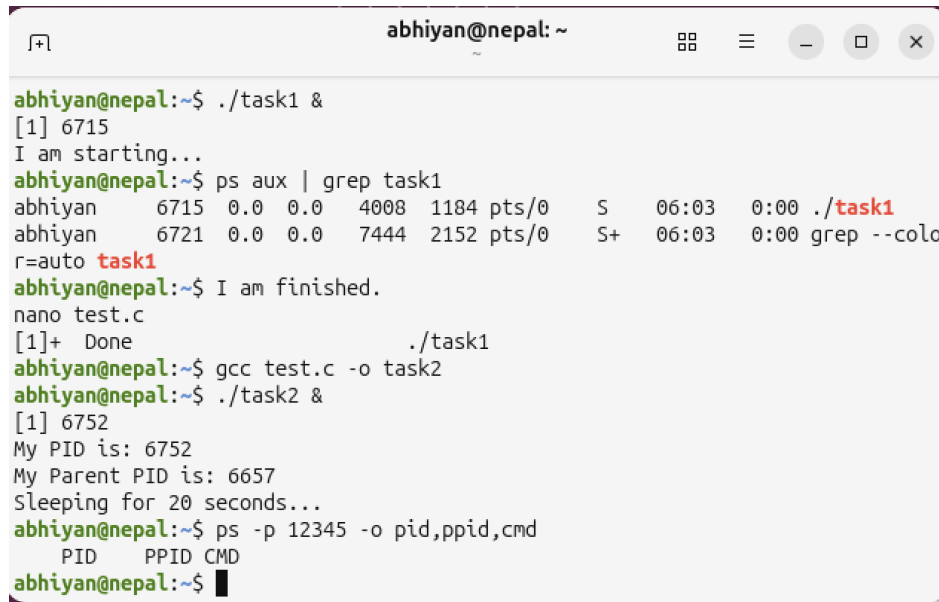
```
abhiyan@nepal: ~
abhiyan@nepal:~$ nano test.c
abhiyan@nepal:~$ gcc test.c -o task1
abhiyan@nepal:~$ ./task1 &
[1] 6715
I am starting...
abhiyan@nepal:~$ ps aux | grep task1
abhiyan    6715  0.0  0.0  4008  1184 pts/0    S   06:03   0:00 ./task1
abhiyan    6721  0.0  0.0  7444  2152 pts/0    S+  06:03   0:00 grep --colo
r=auto task1
abhiyan@nepal:~$ I am finished.
nano test.c
[1]+  Done                  ./task1
abhiyan@nepal:~$ gcc test.c -o task2
abhiyan@nepal:~$ ./task2 &
[1] 6752
My PID is: 6752
My Parent PID is: 6657
Sleeping for 20 seconds...
abhiyan@nepal:~$
```

Figure 6. task2 reporting PID 6752 and parent PID 6657.

C. Querying a Process with ps -p

`ps -p <pid> -o pid,ppid,cmd` prints only the chosen process and only the chosen columns (PID, parent PID and command). In this run the command was entered with the placeholder PID 12345 rather than the real PID 6752. No process with that ID existed, so `ps` printed just the column header and no row. Using the PID that `task2` actually reported (`ps -p 6752 -o pid,ppid,cmd`) would list one row showing PID 6752, PPID 6657 and `./task2`. The job then finished on its own and the shell reported `[1]+ Done ./task2`.

```
ps -p 12345 -o pid,ppid,cmd
```



```
abhiyan@nepal: ~  
abhiyan@nepal:~$ ./task1 &  
[1] 6715  
I am starting...  
abhiyan@nepal:~$ ps aux | grep task1  
abhiyan      6715  0.0  0.0  4008  1184 pts/0    S   06:03   0:00 ./task1  
abhiyan      6721  0.0  0.0  7444  2152 pts/0    S+  06:03   0:00 grep --colo  
r=auto task1  
abhiyan@nepal:~$ I am finished.  
nano test.c  
[1]+  Done                  ./task1  
abhiyan@nepal:~$ gcc test.c -o task2  
abhiyan@nepal:~$ ./task2 &  
[1] 6752  
My PID is: 6752  
My Parent PID is: 6657  
Sleeping for 20 seconds...  
abhiyan@nepal:~$ ps -p 12345 -o pid,ppid,cmd  
PID      PPID CMD  
abhiyan@nepal:~$
```

Figure 7. `ps -p` with a non-existent PID (12345) returns only the header line.

Summary: `getpid()` and `getppid()` returned the process's own ID (6752) and its parent shell's ID (6657). The `ps -p` query returned no row because the PID used did not belong to a running process, which shows that `ps` only lists processes that currently exist.

IV. Task 3 — Return Values and Exit Status

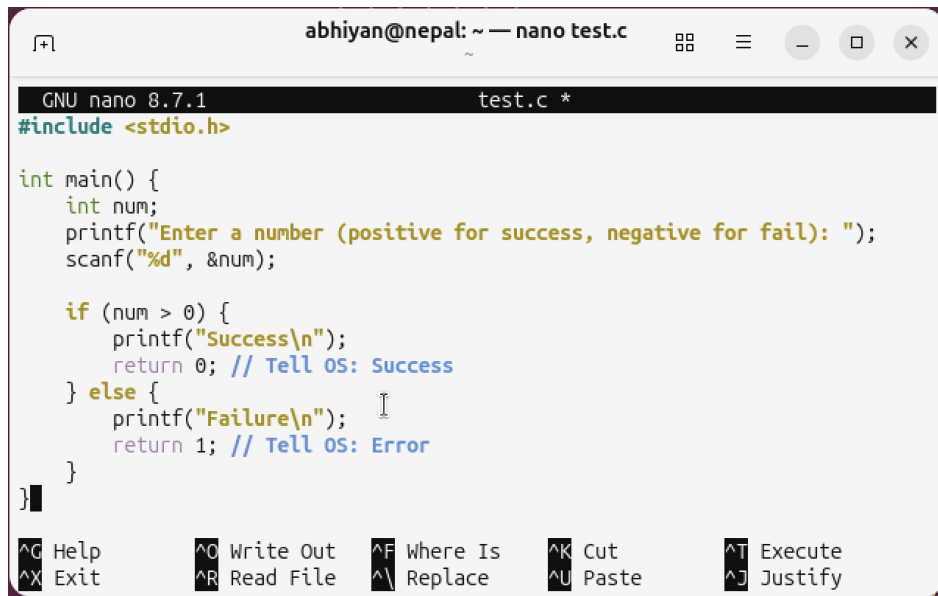
A. The Program

The third program reads an integer. A positive number prints "Success" and returns 0; any other number prints "Failure" and returns 1. The return value of `main()` is handed to the shell as the exit status, where 0 means success and non-zero means an error.

```
#include <stdio.h>

int main() {
    int num;
    printf("Enter a number (positive for success, negative for fail): ");
    scanf("%d", &num);

    if (num > 0) {
        printf("Success\n");
        return 0; // Tell OS: Success
    } else {
        printf("Failure\n");
        return 1; // Tell OS: Error
    }
}
```



```
GNU nano 8.7.1 test.c *
#include <stdio.h>

int main() {
    int num;
    printf("Enter a number (positive for success, negative for fail): ");
    scanf("%d", &num);

    if (num > 0) {
        printf("Success\n");
        return 0; // Tell OS: Success
    } else {
        printf("Failure\n");
        return 1; // Tell OS: Error
    }
}
```

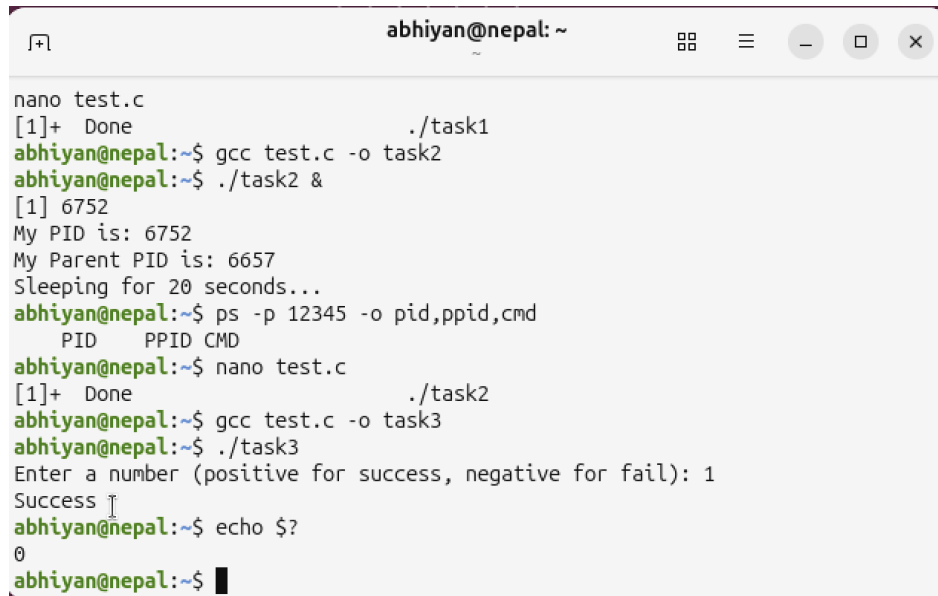
Help Write Out Where Is Cut Execute
Exit Read File Replace Paste Justify

Figure 8. test.c for Task 3 in nano, returning 0 or 1 depending on the input.

B. Testing Both Outcomes

The program was compiled to task3 and run twice, checking the exit status with `echo $?` straight after each run. Entering 1 produced “Success” and an exit status of 0.

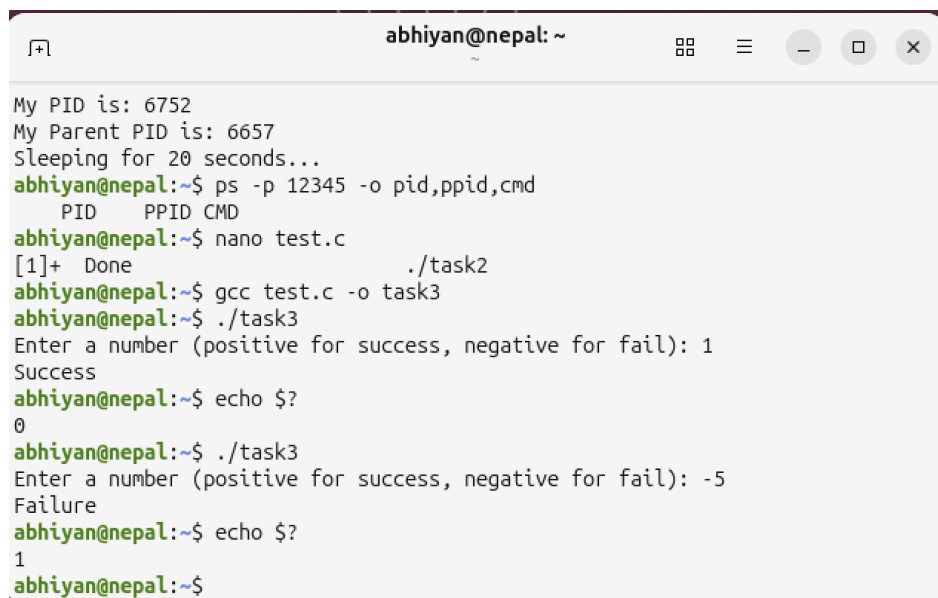
```
gcc test.c -o task3
./task3
echo $?
```

A terminal window titled 'abhiyan@nepal: ~' showing the execution of a C program. The user runs 'nano test.c', then './task1', then 'gcc test.c -o task2', and finally './task2 &'. The program outputs 'My PID is: 6752', 'My Parent PID is: 6657', and 'Sleeping for 20 seconds...'. The user then runs 'ps -p 12345 -o pid,ppid,cmd', which shows the process. Next, the user runs 'nano test.c' again, then './task2', then 'gcc test.c -o task3', and finally './task3'. The program prompts 'Enter a number (positive for success, negative for fail): 1' and the user enters '1'. The program outputs 'Success'. The user then runs 'echo \$?', which outputs '0'.

```
abhiyan@nepal: ~
nano test.c
[1]+  Done                  ./task1
abhiyan@nepal:~$ gcc test.c -o task2
abhiyan@nepal:~$ ./task2 &
[1] 6752
My PID is: 6752
My Parent PID is: 6657
Sleeping for 20 seconds...
abhiyan@nepal:~$ ps -p 12345 -o pid,ppid,cmd
  PID   PPID  CMD
abhiyan@nepal:~$ nano test.c
[1]+  Done                  ./task2
abhiyan@nepal:~$ gcc test.c -o task3
abhiyan@nepal:~$ ./task3
Enter a number (positive for success, negative for fail): 1
Success
abhiyan@nepal:~$ echo $?
0
abhiyan@nepal:~$
```

Figure 9. Input 1 prints Success and echo \$? reports 0.

Entering -5 produced “Failure” and an exit status of 1.

A terminal window titled 'abhiyan@nepal: ~' showing the execution of a C program. The user runs 'ps -p 12345 -o pid,ppid,cmd', then 'nano test.c', then './task2', then 'gcc test.c -o task3', and finally './task3'. The program prompts 'Enter a number (positive for success, negative for fail): 1' and the user enters '1'. The program outputs 'Success'. The user then runs 'echo \$?', which outputs '0'. Next, the user runs './task3' again. The program prompts 'Enter a number (positive for success, negative for fail): -5' and the user enters '-5'. The program outputs 'Failure'. The user then runs 'echo \$?', which outputs '1'.

```
abhiyan@nepal: ~
My PID is: 6752
My Parent PID is: 6657
Sleeping for 20 seconds...
abhiyan@nepal:~$ ps -p 12345 -o pid,ppid,cmd
  PID   PPID  CMD
abhiyan@nepal:~$ nano test.c
[1]+  Done                  ./task2
abhiyan@nepal:~$ gcc test.c -o task3
abhiyan@nepal:~$ ./task3
Enter a number (positive for success, negative for fail): 1
Success
abhiyan@nepal:~$ echo $?
0
abhiyan@nepal:~$ ./task3
Enter a number (positive for success, negative for fail): -5
Failure
abhiyan@nepal:~$ echo $?
1
abhiyan@nepal:~$
```

Figure 10. Input -5 prints Failure and echo \$? reports 1, after the earlier run.

Summary: The exit status seen by the shell matched the value returned from `main()`: 0 for the positive input and 1 for the negative input. This is the same mechanism seen in Lab 2, where returning 50 produced an exit status of 50.

V. Task 4 — Reading User Input

The fourth program declares a character array, prompts for a name, reads it with `scanf()`, and greets the user. The `%s` format reads characters up to the next whitespace, so it captures a single word; a name

containing a space would be cut off at the space. The array holds 50 characters, so a longer word would overflow it. A width limit such as %49s avoids that.

```
#include <stdio.h>

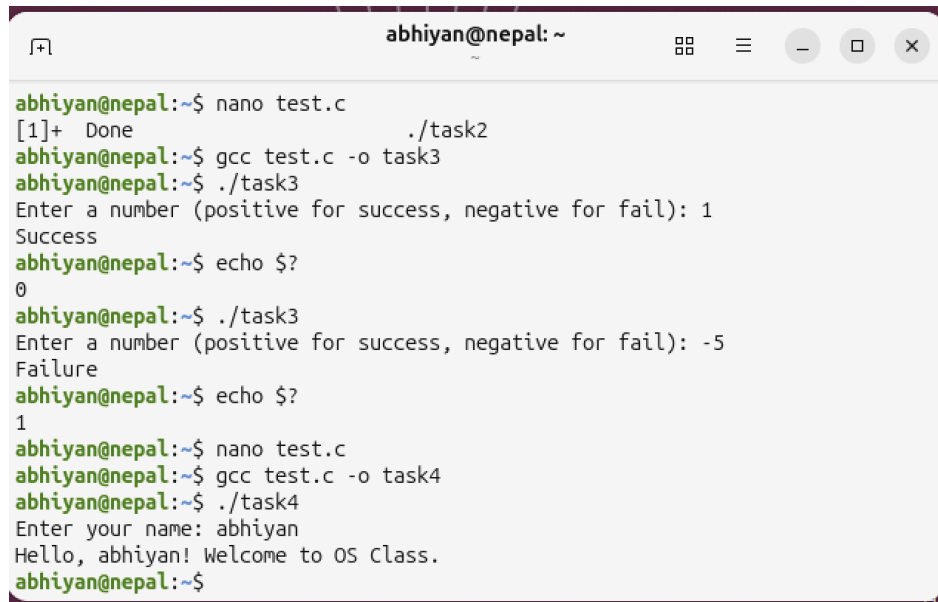
int main() {
    char name[50];
    printf("Enter your name: ");
    // Read string from standard input
    scanf("%s", name);
    printf("Hello, %s! Welcome to OS Class.\n", name);
    return 0;
}
```



Figure 11. test.c for Task 4 in nano; the save prompt appears as the editor is closed.

After compiling to task4, entering “abhiyan” produced the greeting below.

```
gcc test.c -o task4
./task4
```



```
abhiyan@nepal: ~  
abhiyan@nepal:~$ nano test.c  
[1]+  Done                  ./task2  
abhiyan@nepal:~$ gcc test.c -o task3  
abhiyan@nepal:~$ ./task3  
Enter a number (positive for success, negative for fail): 1  
Success  
abhiyan@nepal:~$ echo $?  
0  
abhiyan@nepal:~$ ./task3  
Enter a number (positive for success, negative for fail): -5  
Failure  
abhiyan@nepal:~$ echo $?  
1  
abhiyan@nepal:~$ nano test.c  
abhiyan@nepal:~$ gcc test.c -o task4  
abhiyan@nepal:~$ ./task4  
Enter your name: abhiyan  
Hello, abhiyan! Welcome to OS Class.  
abhiyan@nepal:~$
```

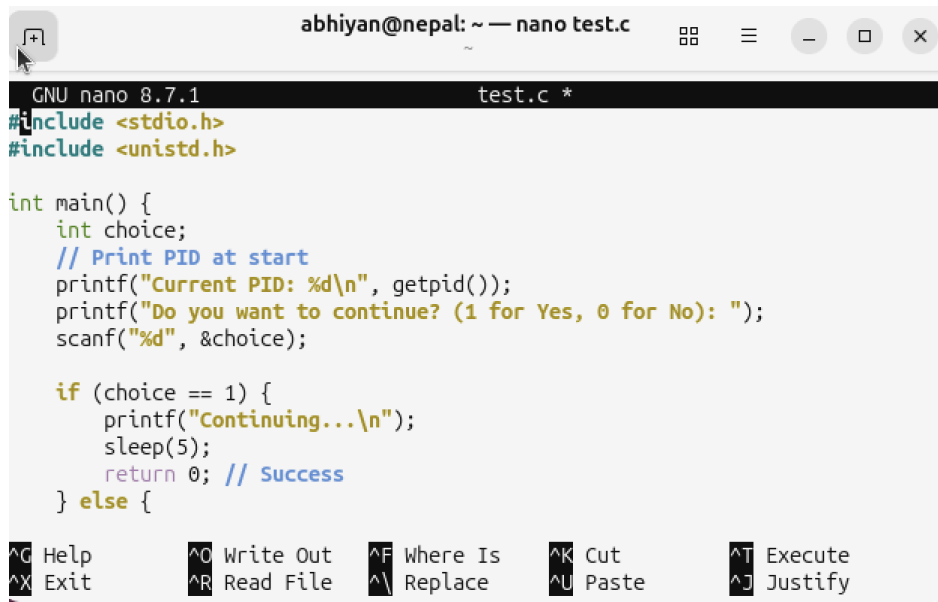
Figure 12. task4 reading a name and printing the greeting.

Summary: `scanf()` with `%s` read the typed name into the character array, and `printf()` displayed it back inside the greeting.

VI. Task 5 — PID, Input and Exit Status Together

The final program combines the earlier ideas. It prints its own PID at the start, asks whether to continue, and then exits with a status that depends on the answer: entering 1 prints “Continuing...”, waits 5 seconds and returns 0 (success); entering anything else prints “Exiting...” and returns 1.

```
#include <stdio.h>  
#include <unistd.h>  
  
int main() {  
    int choice;  
    // Print PID at start  
    printf("Current PID: %d\n", getpid());  
    printf("Do you want to continue? (1 for Yes, 0 for No): ");  
    scanf("%d", &choice);  
  
    if (choice == 1) {  
        printf("Continuing...\n");  
        sleep(5);  
        return 0; // Success  
    } else {  
        printf("Exiting...\n");  
        return 1;  
    }  
}
```



```
GNU nano 8.7.1 test.c *
#include <stdio.h>
#include <unistd.h>

int main() {
    int choice;
    // Print PID at start
    printf("Current PID: %d\n", getpid());
    printf("Do you want to continue? (1 for Yes, 0 for No): ");
    scanf("%d", &choice);

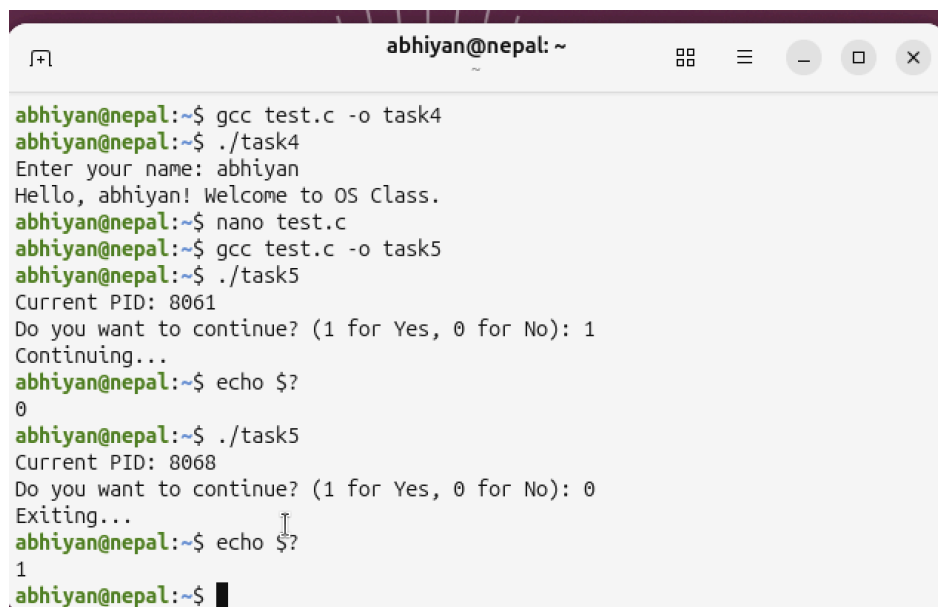
    if (choice == 1) {
        printf("Continuing...\n");
        sleep(5);
        return 0; // Success
    } else {

```

Figure 13. test.c for Task 5 in nano (first part of the program).

task5 was run twice. Each run printed a different PID (8061, then 8068), because every launch creates a new process. With input 1 the program continued and echo \$? reported 0; with input 0 it exited early and echo \$? reported 1.

```
gcc test.c -o task5
./task5
echo $?
```



```
abhiyan@nepal:~$ gcc test.c -o task4
abhiyan@nepal:~$ ./task4
Enter your name: abhiyan
Hello, abhiyan! Welcome to OS Class.
abhiyan@nepal:~$ nano test.c
abhiyan@nepal:~$ gcc test.c -o task5
abhiyan@nepal:~$ ./task5
Current PID: 8061
Do you want to continue? (1 for Yes, 0 for No): 1
Continuing...
abhiyan@nepal:~$ echo $?
0
abhiyan@nepal:~$ ./task5
Current PID: 8068
Do you want to continue? (1 for Yes, 0 for No): 0
Exiting...
abhiyan@nepal:~$ echo $?
1
abhiyan@nepal:~$
```

Figure 14. Two runs of task5: input 1 gives exit status 0, input 0 gives exit status 1.

Summary: task5 showed that each run gets its own PID and that the program's exit status follows the branch taken: 0 on the continue path and 1 on the exit path.

VII. Knowledge Check Questions

Q: What does the & symbol do at the end of a command, and how was it used in this lab?

A: A trailing & tells the shell to run the command as a background job. The shell prints the job number and PID, for example [1] 6715, and returns the prompt immediately instead of waiting for the program to finish. In this lab it was used to start task1 and task2 so that ps could be run while they were still executing. The jobs command lists background jobs, and fg brings one back to the foreground.

Q: What is the difference between a PID and a PPID? What were they for task2?

A: The PID is the unique number the operating system assigns to a process, while the PPID is the PID of the process that created it (its parent). For task2 the PID was 6752 and the PPID was 6657, which was the shell the program was launched from.

Q: Why did ps aux | grep task1 show two lines, and what do the S and S+ states mean?

A: The first line was the task1 process itself. The second was the grep command, whose own command line contains the text task1 and therefore matches the search. S means the process is sleeping, waiting for an event or timer, which is where task1 spent its time inside sleep(). The + in S+ marks a process in the foreground process group of the terminal, which is why grep was S+ while the background task1 was plain S.

Q: Why did ps -p 12345 -o pid,ppid,cmd print only a header line?

A: The PID 12345 did not belong to any running process, so there was no row to display and ps printed only the column names. The real PID reported by task2 was 6752, and ps -p 6752 -o pid,ppid,cmd would have shown that process.

Q: What does echo \$? display, and why is it used right after running a program?

A: echo \$? prints the exit status of the most recently completed command. It has to be run immediately after the program, because the next command overwrites the value. In Task 3 and Task 5 it showed 0 for the success path and 1 for the failure path, matching the values returned by main().

Q: Write a C program that prints its own process ID and then exits with status 3.

A: getpid() from unistd.h returns the PID, and returning 3 from main() sets the exit status:

```
#include <stdio.h>
#include <unistd.h>

int main() {
    printf("My PID is: %d\n", getpid());
    return 3;
}
```

After running it, echo \$? would print 3.

VIII. Lab Tasks Completed

1. Wrote a C program that runs for 30 seconds, ran it in the background with &, and inspected it with ps aux | grep (Figures 1–4).

2. Printed a process's PID and parent PID with `getpid()` and `getppid()`, and queried a process with `ps -p` (Figures 5–7).
3. Returned 0 and 1 from `main()` depending on user input and verified each exit status with `echo $?` (Figures 8–10).
4. Read a string from the user with `scanf()` and printed a personalised greeting (Figures 11–12).
5. Combined PID printing, user input and exit status in one program and tested both paths (Figures 13–14).

IX. Conclusion

This lab moved from single programs to programs seen as processes. Running `task1` in the background showed that the shell can keep working while a job runs, and `ps` made that job visible with its PID and sleeping state. `task2` showed how a process can find its own ID and its parent's, and the `ps -p` query with an unused PID showed that only live processes are listed. Tasks 3 and 5 reinforced that the value returned from `main()` becomes the exit status the shell reads with `echo $?`, and Task 4 covered basic string input with `scanf()`.

GitHub link: https://github.com/abhiyan0/batch38_ai_OS_abhiyansharma.git